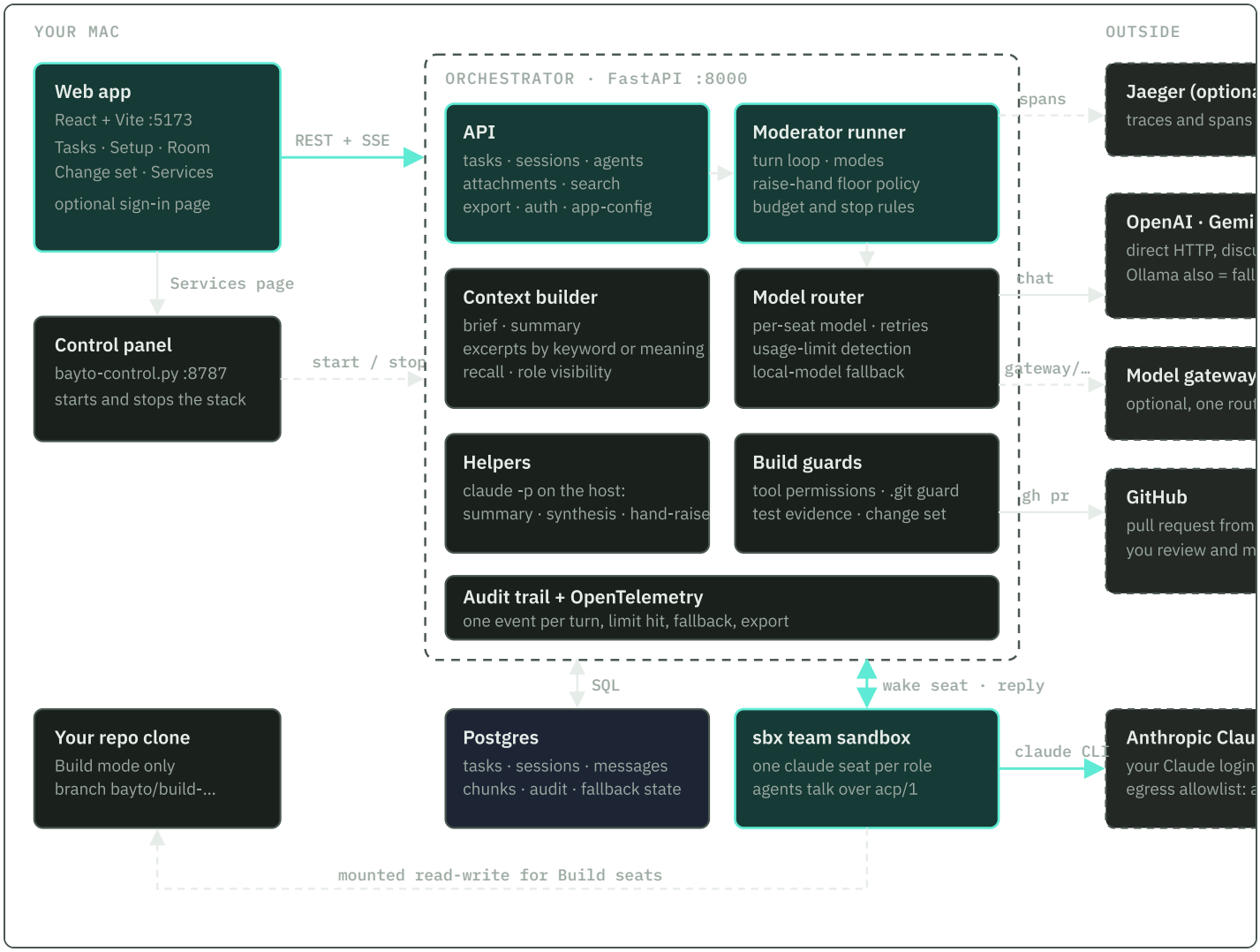
BAYTO · main after PR #211 · 2026-10-25

Bayto architecture

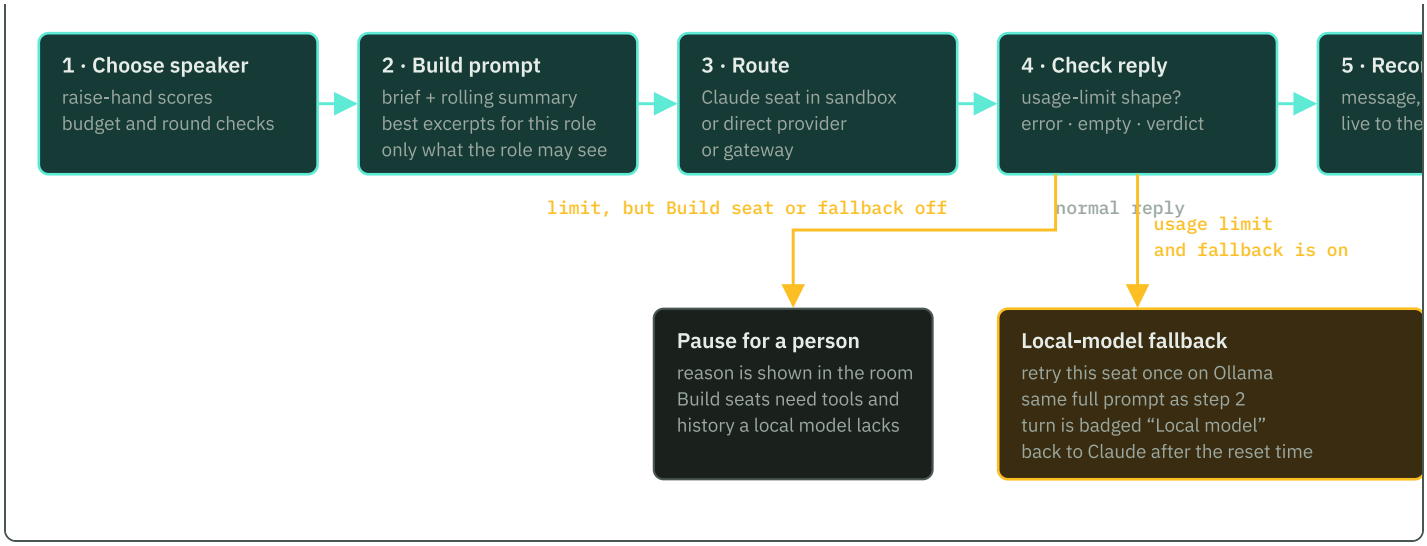
A local-first app where several AI agents take turns on one task. A web page talks to one orchestrator; the orchestrator decides who speaks, builds each agent's prompt, sends the turn to a model, and records everything in Postgres.

1 · What runs where



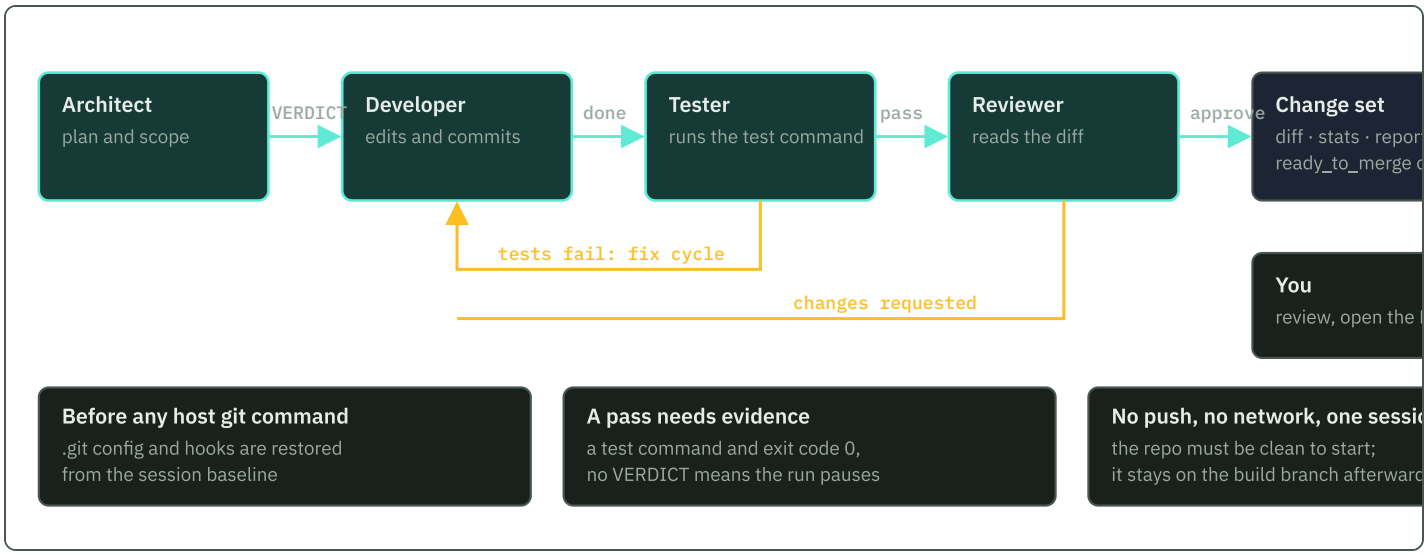
The browser never talks to a model. Every turn goes web app → orchestrator → a model, and every result is written to Postgres before it is streamed back. Discussion seats can use other providers; Build seats run only in the sandbox.

2 · One turn, including the usage-limit path



The prompt is built once and reused, so a fallback seat sees the same context Claude would have. The amber path only exists when the task opts in; otherwise a limit pauses the session instead of changing the model quietly.

3 · Build mode: from brief to pull request



Each role ends with a `VERDICT:` line that routes the next step. A fix cycle is bounded, and anything unroutable stops the session for a person instead of looping.

Design choices worth knowing

Local first, no accounts

One orchestrator, one Postgres, your own Claude login. Sign-in is an optional shared token for when the port is reachable by others.

Context is chosen per seat

Each seat gets the excerpts most relevant to its role and the current question, filtered by who may see each attachment, within a fixed budget.

Everything is recorded

Changes need a person

Everything is recorded

Messages, usage, stop reasons, fallbacks and exports are stored and audited, so a session can be exported as a decision record.

Changes need a person

Build never pushes or merges. Output is a branch, a change set and a pull request that you open.